

ANN for Classification Using Keras

MACHINE LEARNING VS DEEP LEARNING

A Practical Study Guide for Data Science & AI Beginners

Predicting Survival using the Seaborn Titanic Dataset
Focus: Practical Implementation & Code Explanation

SECTION 1: INTRODUCTION

What is Classification?

In Machine Learning, **Classification** is a predictive modeling technique used when the target variable is a category or a discrete class. Unlike Regression, where we predict a continuous number (like \$15.50),

in Classification we predict labels. The most common type is **Binary Classification**, where there are only two possible outcomes: True or False, Yes or No, 1 or 0.

Why Classification Problems Exist in Real Life?

Classification is used to make definitive categorical decisions. Common real-life examples include:

- **Healthcare:** Predicting whether a tumor is Malignant (Cancerous) or Benign (Safe) based on medical test results.
- **Finance:** Determining if a credit card transaction is Fraudulent or Legitimate.
- **Email:** Filtering incoming emails into Spam or Inbox.
- **History/Our Project:** Predicting if a passenger Survived (1) or Did Not Survive (0) the Titanic disaster based on their age and ticket fare.

Why Neural Networks Can Be Used for Classification?

Artificial Neural Networks (ANNs) are exceptional at classification. By adding a special mathematical function (called a Sigmoid activation) to the final output neuron, the network squishes its predictions into a probability score between 0.0 and 1.0. If the score is above 0.5, the network classifies it as class 1; if below, class 0. Deep Neural Networks can learn highly complex, non-linear boundaries between categories that simple algorithms cannot see.

SECTION 2: DATASET

What is the Titanic Dataset?

The **Titanic dataset** is a famous dataset built into the Seaborn library. It contains demographic and travel information about the passengers aboard the RMS Titanic, along with a record of whether they survived the tragic sinking.

Explanation of Columns

When you load the dataset, you will see several columns. Here are the key ones:

- **survived**: Did the passenger survive? 1 = Yes, 0 = No (Categorical/Target).

- **age** : The age of the passenger in years (Numeric).
- **fare** : The price the passenger paid for their ticket (Numeric).
- **class** : The ticket class (First, Second, Third).
- **sex** : Gender of the passenger (Male, Female).

Sample Dataset Output

age	fare	survived
22.0	7.2500	0
38.0	71.2833	1
26.0	7.9250	1

Why Choose age and fare as Input Features?

We choose **age** and **fare** because they are numerical. In the context of the Titanic, age played a role in survival (e.g., "women and children first"), and fare strongly correlates with socio-economic status (First Class vs Third Class), which also impacted survival rates. Using only numerical columns keeps the beginner code clean by avoiding categorical encoding.

Why Choose survived as the Target Variable?

The **survived** column is the perfect binary classification target. It contains only 0s and 1s. We want the AI to learn the patterns in age and ticket price to classify whether a person lived or died.

SECTION 3: IMPORT LIBRARIES

Here is the complete import block needed for this project.

```
import pandas as pd
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
```

Data Handling & Visualization

- `import pandas as pd`

Why it is used: Provides the DataFrame structure for data manipulation. It lets us drop missing values and select columns.

Without it: Data manipulation in pure Python is extremely difficult and slow.

- `import seaborn as sns`

Why it is used: We use it to download the `titanic` dataset directly via

`sns.load_dataset('titanic')`.

Without it: We would need to manually find and download a CSV file.

Machine Learning (Scikit-Learn)

- `from sklearn.model_selection import train_test_split`

Why it is used: Shuffles and divides our data into a Training Set and a Testing Set to prevent cheating (data leakage).

- `from sklearn.preprocessing import StandardScaler`

Why it is used: Scales data so features like Age (0-80) and Fare (0-500) have the same mathematical weight.

Without it: The neural network might think `fare` is more important just because the numbers are bigger.

- `from sklearn.linear_model import LogisticRegression`

Why it is used: Despite the word "Regression" in its name, Logistic Regression is the classic Machine Learning algorithm for **Classification**.

Without it: We wouldn't have a baseline ML model to compare against our Deep Learning model.

- `from sklearn.metrics import accuracy_score, confusion_matrix`

Why it is used: To evaluate our model. Accuracy gives a percentage of correct guesses. The confusion matrix shows exactly where the model got confused (e.g., predicting survival when they actually died).

Deep Learning (Keras/TensorFlow)

- `from tensorflow.keras.models import Sequential`

Why it is used: Initializes a linear stack of layers for our Artificial Neural Network.

- `from tensorflow.keras.layers import Dense`

Why it is used: Adds fully connected artificial neurons to our Sequential model.

SECTION 4: FEATURE SELECTION

```
# Load the dataset
df = sns.load_dataset('titanic')

# Keep only necessary columns and drop rows with missing values
df = df[['age', 'fare', 'survived']].dropna()

# Separate Inputs (X) and Output (y)
X = df[['age', 'fare']]
y = df['survived']
```

Explain Line by Line

- `df[['age', 'fare', 'survived']].dropna()` : Datasets often have missing information (NaN). If we pass missing values into a neural network, it crashes. `dropna()` simply deletes any row where age or fare is missing.
- `X = df[['age', 'fare']]` : We create a 2D DataFrame `X` containing our **Inputs/Features**. These are the clues the AI will use.
- `y = df['survived']` : We create a 1D Series `y` containing our **Output/Target**. These are the actual answers (1 or 0) we want to predict.

SECTION 5: TRAIN TEST SPLIT

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42
)
```

Explanation of Parameters

- `test_size=0.2` : Reserves 20% of the passenger data as a hidden exam to test the model later. The other 80% is used for studying (training).
- `random_state=42` : Ensures the random shuffle happens exactly the same way every time we run the script, guaranteeing reproducible results.

Why Training and Testing Data Are Needed?

If we tested the model on the exact same data it trained on, it might just memorize the answers rather than learning the actual underlying patterns (a problem known as Overfitting). By keeping a hidden `X_test` set, we can prove the model has genuinely learned how to classify survival based on age and fare, even for passengers it has never seen before.

SECTION 6: FEATURE SCALING

```
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

Explain Line by Line

- `scaler = StandardScaler()` : Creates the scaling tool.
- `scaler.fit_transform(X_train)` : Calculates the mean and standard deviation of Age and Fare in the training set (fit), and then applies the transformation to standardize them around zero.

- `scaler.transform(X_test)` : Applies the exact same scaling rules learned from the training data to the testing data.

Why Scaling is Important

Age ranges from roughly 0 to 80. Fare can range from 0 to over 500. Without scaling, a Neural Network's gradients will struggle to converge during training because the `fare` input will dominate the mathematical calculations. Scaling ensures that `age` and `fare` are put on a level playing field.

SECTION 7: MACHINE LEARNING MODEL

Let's build a baseline traditional Machine Learning model using Logistic Regression.

```
# Initialize the classification model
ml_model = LogisticRegression()

# Train the model on the training data
ml_model.fit(X_train_scaled, y_train)

# Generate predictions on the test data
ml_predictions = ml_model.predict(X_test_scaled)
```

Explain the Code

- `fit()` : The algorithm examines `X_train_scaled` and `y_train` to draw a mathematical boundary line separating the survivors from the deceased.
- `predict()` : Takes the unseen `X_test_scaled` and outputs a hard class label (either a solid 0 or a solid 1) based on which side of the boundary line the passenger falls.

How Logistic Regression Learns and Predicts

Logistic Regression works similarly to Linear Regression but passes its final output through an S-shaped curve called the **Sigmoid Function**. This function squashes any mathematical output into a probability between 0 and 1. If the probability is greater than 0.5, it predicts class 1 (Survived); otherwise, it predicts class 0 (Died).

Architecture Diagram

```
[Inputs: Age, Fare] ---> Linear Math ---> [Sigmoid Function] --->
Probability (0 to 1) ---> Class (0 or 1)
```

SECTION 8: MODEL EVALUATION

```
# Calculate Accuracy
ml_accuracy = accuracy_score(y_test, ml_predictions)

# Calculate Confusion Matrix
ml_cm = confusion_matrix(y_test, ml_predictions)

print(f"ML Accuracy: {ml_accuracy * 100:.2f}%")
print("Confusion Matrix:\n", ml_cm)
```

What is Accuracy Score?

Accuracy simply measures the percentage of correct predictions.

Interpretation: An accuracy of 70% means that out of 100 passengers, the model correctly guessed whether 70 of them lived or died.

What is a Confusion Matrix?

Accuracy alone can be misleading. A Confusion Matrix breaks down exactly *how* the model was right and wrong into 4 categories:

- **True Positives (TP):** Model predicted survived, and they actually survived.
- **True Negatives (TN):** Model predicted died, and they actually died.
- **False Positives (FP):** Model predicted survived, but they actually died (False Alarm).
- **False Negatives (FN):** Model predicted died, but they actually survived (Missed Detection).

SECTION 9: ANN WITH SINGLE NEURON

Moving to Deep Learning, the simplest neural network for classification is a single neuron with a Sigmoid activation function. Mathematically, this is identical to Logistic Regression!

```
# Build the model architecture
single_neuron_model = Sequential([
    Dense(1, input_dim=2, activation='sigmoid')
])
```

Explain the Architecture

- `Sequential` : A container to build the model layer by layer.
- `Dense(1)` : A single fully connected artificial neuron.
- `input_dim=2` : Expecting 2 inputs (Age and Fare).
- `activation='sigmoid'` : **Crucial Difference from Regression!** Instead of outputting raw linear numbers, the sigmoid function catches the output of the neuron and forces it to be a decimal between 0.0 and 1.0, representing the probability of survival.

Why this ANN Behaves Similar to Logistic Regression

Because the single neuron performs a linear combination of inputs ($\text{Weights} \times \text{Inputs} + \text{Bias}$) and then applies a Sigmoid activation, it is executing the exact same mathematical formula as Scikit-Learn's Logistic Regression under the hood.

SECTION 10: COMPILE MODEL

```
single_neuron_model.compile(
    optimizer='adam',
    loss='binary_crossentropy',
    metrics=['accuracy']
)
```

Explanation

- `compile()` : Sets up the learning process.
- `optimizer='adam'` : The algorithm that adjusts weights based on errors. Adam is highly efficient.
- `loss='binary_crossentropy'` : **Crucial Difference from Regression!** We cannot use MSE here. Binary Crossentropy is a specialized math function designed to calculate errors in probability predictions for 2-class problems. It heavily penalizes the model if it is highly confident about a wrong answer.
- `metrics=['accuracy']` : Tells Keras to print out the classification accuracy as the model trains, making it easy for humans to monitor.

SECTION 11: TRAIN MODEL

```
history = single_neuron_model.fit(  
    X_train_scaled,  
    y_train,  
    epochs=50,  
    verbose=1  
)
```

Explanation

- `epochs=50` : The model will read the entire training dataset 50 times to incrementally adjust its weights and improve its accuracy.

The Training Process: Step-by-Step

1. **Forward Propagation:** Age and Fare are multiplied by the neuron's weights. The Sigmoid function spits out a probability (e.g., 0.80 or 80% chance of survival).
2. **Loss Calculation:** The model checks the true answer (e.g., passenger actually survived, target = 1). Binary Crossentropy calculates the penalty based on how far 0.80 is from 1.0.
3. **Backpropagation:** The math works backward to find out which weight caused the 0.20 error margin.
4. **Weight Update:** The Adam Optimizer adjusts the weights so next time, the probability is closer to 1.0.

SECTION 12: PREDICTION

```
# ANN predicts probabilities (0 to 1), so we convert them to 0 or 1 classes
dl_single_probs = single_neuron_model.predict(X_test_scaled)
dl_single_predictions = (dl_single_probs > 0.5).astype(int)
```

What Prediction Means Here

Unlike Scikit-Learn which automatically outputs a 0 or 1, Keras Deep Learning outputs raw probabilities (e.g., `[0.12, 0.99, 0.45]`). To evaluate accuracy, we apply a **Threshold**. We write code saying: "If the probability is greater than 0.5, count it as a 1 (Survived). Otherwise, convert it to a 0 (Died)." The `.astype(int)` turns True/False into 1/0.

SECTION 13: MULTI-LAYER ANN

To learn complex, non-linear classification boundaries, we add Hidden Layers using the ReLU activation, while keeping the final output layer as Sigmoid.

```
multi_layer_model = Sequential([
    Dense(8, activation='relu', input_dim=2),
    Dense(4, activation='relu'),
    Dense(1, activation='sigmoid') # Final layer MUST be sigmoid for binary classification
])

multi_layer_model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
multi_layer_model.fit(X_train_scaled, y_train, epochs=50, verbose=0)

dl_multi_probs = multi_layer_model.predict(X_test_scaled)
dl_multi_preds = (dl_multi_probs > 0.5).astype(int)
```

Explain Every Layer Individually

- `Dense(8, activation='relu')`: The first hidden layer. It looks at Age and Fare and creates 8 new complex internal features. The **ReLU** activation function handles non-linear patterns.
- `Dense(4, activation='relu')`: The second hidden layer compresses those 8 features down to 4 highly refined patterns.

- `Dense(1, activation='sigmoid')` : The final output layer takes the results of the hidden layers and squashes them into a final 0.0 to 1.0 probability of survival.

SECTION 14: COMPARISON

Feature	Logistic Regression (ML)	Single Neuron ANN (DL)	Multi-Layer ANN (DL)
Architecture	Mathematical Equation	1 Neuron Output Layer	Input → Hidden Layers → Output
Output Activation	Built-in Sigmoid	Manual Sigmoid specified	Hidden: ReLU / Output: Sigmoid
Loss Function	Log Loss	Binary Crossentropy	Binary Crossentropy
Learning Capability	Linear Boundaries	Linear Boundaries	Complex, Non-linear Boundaries

SECTION 15: COMPLETE PROJECT CODE

```
# 1. Import all required libraries
import pandas as pd
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense

# 2. Load and clean the dataset
df = sns.load_dataset('titanic')
df = df[['age', 'fare', 'survived']].dropna()

# 3. Separate features (X) and target (y)
X = df[['age', 'fare']]
y = df['survived']

# 4. Split data (80% train, 20% test)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# 5. Scale features
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# --- APPROACH 1: MACHINE LEARNING (LOGISTIC REGRESSION) ---
ml_model = LogisticRegression()
ml_model.fit(X_train_scaled, y_train)
ml_preds = ml_model.predict(X_test_scaled)
print("ML Accuracy:", accuracy_score(y_test, ml_preds))

# --- APPROACH 2: DEEP LEARNING (MULTI-LAYER ANN) ---
dl_model = Sequential([
    Dense(8, activation='relu', input_dim=2),
    Dense(4, activation='relu'),
    Dense(1, activation='sigmoid') # Binary Classification requirement
])

dl_model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
dl_model.fit(X_train_scaled, y_train, epochs=50, verbose=0)

# Predict and apply threshold
dl_probs = dl_model.predict(X_test_scaled)
dl_preds = (dl_probs > 0.5).astype(int)
print("DL Multi-Layer Accuracy:", accuracy_score(y_test, dl_preds))
```

Common Beginner Mistakes

- **Using MSE for Classification:** If you use `loss='mse'` for classification, the neural network will struggle to learn probabilities correctly. Always use `binary_crossentropy` for two-class problems!
- **Forgetting Sigmoid:** If you leave the final layer as `Dense(1)` without `activation='sigmoid'`, the model will spit out raw numbers (like -5 or +40) instead of a probability between 0 and 1, completely ruining the classification process.
- **Not Thresholding DL Predictions:** Keras predicts probabilities. If you try to run `accuracy_score()` directly on probabilities, Scikit-Learn will throw an error. You must convert probabilities to 0s and 1s using `(probs > 0.5).astype(int)` first.

SECTION 16: CONCLUSION

What We Learned

In this guide, we explored Binary Classification using the Titanic survival dataset. We saw how Logistic Regression serves as the traditional Machine Learning baseline, and how a single Artificial Neuron with a Sigmoid activation mimics it perfectly. We then built a full Deep Learning Multi-Layer ANN, utilizing ReLU activations for hidden layers and Sigmoid for the output, governed by the Binary Crossentropy loss function.

Why Hidden Layers Matter

Classification is rarely as simple as drawing a straight line. Hidden layers allow the neural network to warp, bend, and draw complex circular or zigzag boundaries around data points, allowing it to classify highly intricate patterns that standard Logistic Regression would fail to separate.

When to Use What?

Use Logistic Regression (ML) when:

- Your dataset is small to medium-sized.
- You only have simple numerical or categorical features.
- You need high interpretability.

Use Multi-Layer ANN (DL) when:

- Your dataset is massive (hundreds of thousands of rows).
- The boundary between categories is highly non-linear.
- You are working with image, text, or audio classification.